

— A PARADIGM SHIFT • NOT A TOOL ADOPTION

AI-Native Transformation.

从 2x 天花板到 10x 复利 —— 一次范式转移

Human Directs, AI Delivers.

AI-DLC —— 我们的方法论

Xiaogang Wang
CHIEF ARCHITECT + BUILDER

一条主线,四个 Theme

AI-Native Transformation:问题在哪 → 怎么管理 → 怎么造得快 → 怎么守得住



主题 1 · DEFINE THE PROBLEM

定义问题

为什么还是 <2x?—— 瓶颈转移,个人与组织错配

瓶颈没有消失——它转移了

瓶颈没有消失,它转移了 BOTTLENECK SHIFT Agent 解决了中间的 coding,瓶颈移向 两头

Shift Left ←

定义 + 协调

需求 · Spec · 跨角色对齐 —— 新瓶颈 · EE 战场

✓ Solved by Agent

Coding

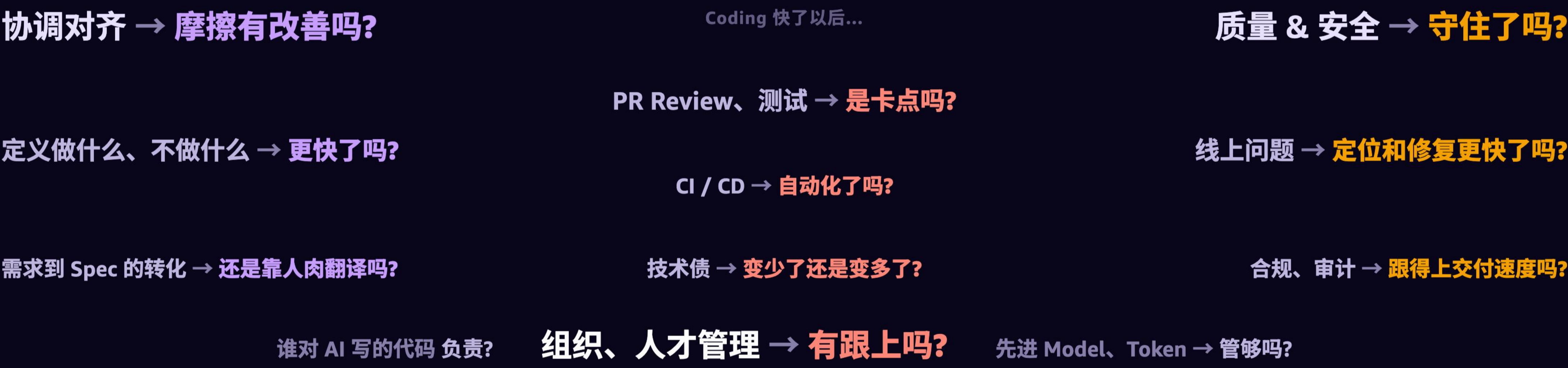
曾经的瓶颈 —— 绝大多数改动由 coding-agent 产出

~30% 链路

Shift Right →

质量 + 安全

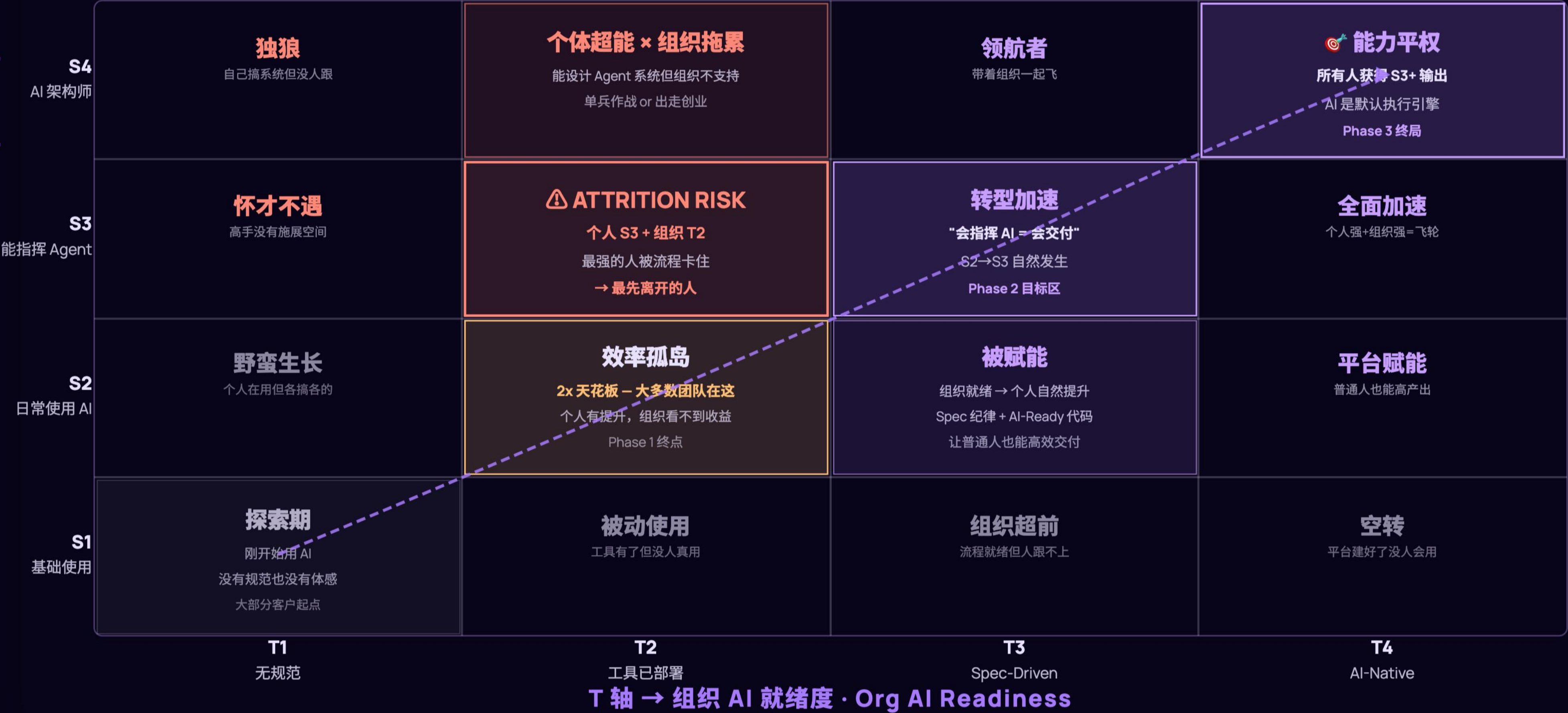
验证 · 回归 · Security —— 新瓶颈 · OE 战场



S × T 诊断矩阵 —— 转型失败是错配

S（纵轴）= 个人 AI 能力 × T（横轴）= 组织就绪度 · 对角线 = 健康推进路径

S 轴 ↑ 个人 AI 能力 · Individual AI Capability



Core Insights

最危险组合
S3+T2: 最强的人最先离开

2x 天花板
T2 = 效率孤岛，Phase 1 结构性终点

T3 解锁 S 跃迁
“会指挥 AI = 会交付”
Org ready → IC 自然升级

T4 = 能力平权
所有人获得 S3 输出
不是淘汰人 — 是赋能

处方
S>T → 推 T
(给先锋者组织支撑)
T>S → 拉 S
(培训 + 让平台赋能个体)

目标：S=T 对角线前进

主题 2 · MANAGEMENT & GUIDELINE

Management & Guideline

系统性的转型路线 —— 五大支柱 · Measurement Metrics · Builder Guideline · 三阶段

五大支柱——是**依赖链**，不是菜单

5 Autonomous
AI 自主判断、执行、验证 —— start from 输入边界 / context 清晰、有安全网、风险可控的地方，比如 Bug Fixing

4 Spec-Driven
Spec 即合约 —— 人和 AI 之间的锚点，所有协调对齐、决策与输出，都是为了可规范 AI 执行的 Spec

----- **Top-down 先建地基 (1,2,3) —— 没有地基 (4-5) 就是空转** -----

3 就绪度 Readiness
清楚知道组织与个人、开发流程各个环节的 AI 就绪度

2 度量 Metrics
量 output / outcome —— input 指标（LoC、PR、AI Adoption）只能当指南针，不能当靶子

1 文化 Culture
AI Agents 是组织的新成员、默认的执行方式 —— 想清楚哪些地方可以交给它们执行

P0 Metrics — Output / Outcome

不是 PR 数量 · AI 生码率 · 工具采纳率 —— 那些是 input, 只能当指南针

OUTPUT 越高越好

1 需求吞吐量
INTAKE VOLUME

2 端到端交付周期
INTAKE-TO-PROD CYCLE-TIME

3 Prod Deploys #
DEPLOYS / ENGINEER / WEEK

4 AI 自主率
TICKETS AUTOMATION RATE

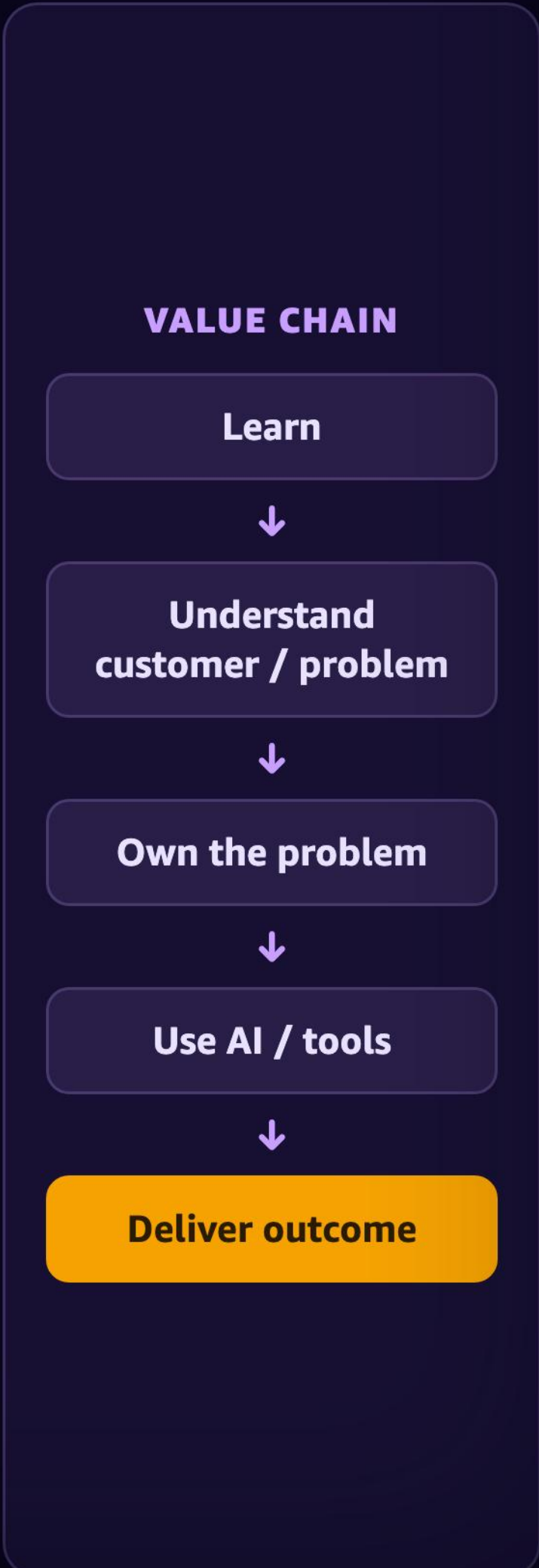
OUTCOME 底线 —— 守不住, 速度再快也没意义

5 交付质量分
TICKET SCORE

6 安全逃逸数
PROD SECURITY FINDINGS

Builder Role Guideline 2026 — 不变的是 LP + EE/OE → Outcome

定义角色的是不变量,不是工具 —— Coding is becoming cheaper, engineering judgment more valuable.



三阶段演进——每阶段改变的是人的角色



Spec-Driven → DDD → Autonomous → Agentic OS 四级递进 · 跳层 = 地基不稳

主题 3 · BUILD FAST

AIDLC Evolution

怎么更快更好地交付（Phase 2 → 3）—— Spec 立规矩,DDD 沉淀判断力,朝 Autonomous 演进

全链条 Spec-Driven —— Spec 是唯一锚点

Engineer 左移决策 · PM / UX 右移验证 —— 握手点减少, **Coordination Tax** 下降



Spec-Driven 规范了组织纪律和 AI 执行——但没改变组织与沉淀判断力

- ✓ SPEC-DRIVEN 做到了

✓ 规范了协作纪律

Spec = 合同，消灭意图漂移与反复对齐。

✓ 规范了 AI 执行

AI 忠于 Spec 交付，行为可预期。
- ✗ 但天花板在这

✗ 没改组织与沉淀

PM / Engineer / QA 仍是顺序筒仓，只是各自用 AI 变快。

✗ 没给 AI 判断力

判断力锁在个人脑里——不 scale、不沉淀，人走即失。

真实项目里的三个痛点 —— 天花板的症状

- 1

Spec 腐烂

写得快,质量与新鲜度跟不上,越改越漂
- 2

Brownfield 冷启动

老库 AI 读不懂,无法 quick understand
- 3

Cross-package 复杂度

隐式依赖,修 A 炸 B,blast radius 看不见

AGENTS.md + System Specs 只解决"导航", 不解决"判断"——配置 ≠ 知识 · 导航 ≠ 理解, Agent 要的是领域知识体系, 不是文件地图。

DDD as Brain —— 让 Domain Expert 的判断力进系统

- ①

Identity

这个产品是什么 · 配置与清单 (AGENTS.md)
- ②

Knowledge

该不该做、能不能动 —— 判断力所在
- ③

Gates

判断变成可执行 hook · 别的 agent 直接继承
- ④

Capabilities

自带「判断→执行→复盘」闭环的 skills

② KNOWLEDGE · 判断力 —— 谁来养，养成什么

PM

PRODUCT

为什么存在 · 不做什么

ENGINEER

TECH

怎么运作 · 什么约定

QA

IMPROVEMENT

踩过的坑 · 反模式

TPM / UXD

PROJECT

在干嘛 · 不能碰什么

组织转型与 AI 赋能，是同一个动作 —— 各角色共养同一个 DDD、共担 Spec / PR，减少摩擦。

DDD 只提供判断力,不做执行 —— 它指向并管理

System Specs

接口契约 · 异常路径 · 架构图 —— 人签字读的那份

Code Graph

domains / flows · file:line · 依赖边 · 爆炸半径

Build & Deploy

构建与部署 —— 执行归工具,判断归 DDD

判断力从专家真实工作里长出来，沉淀进 DDD，被每个 agent 继承 —— judgment 第一次可以 scale，不再人走即失。

Data Agent Sample —— DDD 让 AI 读懂数据

五层架构，判断力（含义 + 谁能看）单独成层 —— Agent 从上往下调用，绕不过去。



DDD Cultivation —— Ontology + 达尔文，越用越准

核心竞争力不是"记住"，是"遗忘" —— 引用 = 自然选择，分类决定半衰期。

✖ 百科全书模式

存下来 · 永不删 · 查询时过滤 —— 越积越肿，没人敢删

vs

✔ 达尔文模式

用则强化 · 不用则衰减 · 最终死亡 —— 引用 = 自然选择

这套 7 类 × 3 层就是一层轻量 Ontology：不上 VectorDB / GraphDB —— 几份 markdown 就够。价值不在"知道得多"，在"能做判断"。

Operational

操作层 · 怎么做

guideline 指导

pitfall 陷阱

process 流程

最易朽

60 天无引用 → 休眠

Cognitive

认知层 · 怎么判断

decision 决策

model 模型

中速

150 天 → 物理剥离

Meta-Cognitive

元认知层 · 怎么想

principle 原则

correction 纠错

★ 常青

被引用即刷新，永不衰减

PRINCIPLE 原则

「删数据前必须先备份」 —— 三年后依然成立，每次改动都会引用，永远 active

GUIDELINE 指导

「这个版本先用 v2 接口」 —— 升级到 v3 后就是错的，自动衰减消失

对比 KARPATHY LLM WIKI ▶ Karpathy 的 LLM Wiki 解决"知识怎么活着不腐烂"；我们在它上面再加一层 —— "知识怎么变成判断力，以及怎么主动遗忘"。

Coding Autonomous Pipeline —— 9 Stages + 3 Gates

需求 → PR-ready 交付。判断力已进系统，AI 才敢自主：该不该做 / 怎么做 / 做完自己验。



DDD 提供 Context → Pipeline 执行 → 对抗找盲点 → Convergence 修复 → REFLECT 写回 DDD。复利循环本身就是产品。

Coding Autonomous——四大机制 · 三道 Gate 缺一不可

底层选择 —— 单 Agent 角色切换，不做 multi-agent 编排

Multi-Agent = 协调税

更多 agent = 判断力更差 · 上下文在 agent 间丢失
拆判断 = 三条裂缝，编排开销吃掉收益

我们的做法：Multi-Skill

一个 agent 切换角色，共享同一份 DDD 判断力
Spawn sub-agent 只做「对抗审查」这类无状态专项

四大机制 —— 让“自主”可靠的四根支柱

1

DDD + SDD + TDD

方法论栈

为什么 · 做什么 · 做完没
缺一 = 盲干 / 漂移 / 质量幻觉

2

对抗 + Convergence

对抗审查 · 质量收敛 ★

独立 Sub-agent 全新上下文找盲点
多层 Push-Ready Gate 同时过 = ship
修复 → 复验 → 收敛 (≤ 3 迭代) ↓ 在 Gate 2 执行



3

Goal-Driven

开放目标

循环执行到 DoD 达标
预算门控 + 卡住检测

4

ESCALATE

自主 ≠ 失控

L0 自动 · L1 确认 · L2 阻塞等人
+ gap report —— 卡住就交回人

三道 Gate —— 缺一不可，逐道收窄

GATE 0

框架 · Framing

诊断先于构建 + M3 skeptic
位置：EVALUATE → THINK 之间

GATE 1

计划 · Plan

Skeptic 攻击计划 + SSA
位置：PLAN 之后 · 写码之前

GATE 2 ★ 对抗审查

构建 · Build

对抗 Sub-agent · 全新上下文
BLOCKING —— 过不了不 ship

深入阅读 · DISCUSSIONS + SKILL 源码

↗ Coding as Black Box

↗ 为什么选 Single-Agent

↗ Autonomous Pipeline v3 双模式

↗ Pipeline Skill 源码

PHASE 3 · 9 STAGES · 3 GATES · 2 MODES

Agentic OS —— 一层知识，多个交付引擎

手脚 DELIVERY ENGINES · 决定交付形态

Autonomous Pipeline

需求 → PR-ready 代码

AI-Ready Repo

交付结构化上下文

Content Engine

消息 → 多形态媒介

Your Engine

你的领域交付场景

大脑 DDD KNOWLEDGE LAYER · INFRASTRUCTURE

Interface Layer

DDD 4-File · 判断力

Code Intelligence

依赖图 · blast_radius

Cultivation

越用越准 · 达尔文衰减

Feed Channels

multiple 知识输入源

FEEDS · 每次 Agent 执行 › Pipeline REFLECT › 用户纠错 › 代码变更 › 外部研究 › 市场信号 › PE Review › 跨项目迁移

底盘 AGENT HARNESS · 决定能做什么

Session Runtime

生命周期 · 流式

Memory & Recall

跨会话记忆

Context Engine

系统提示装配

Hook System

运行时门禁

Skills

可复用能力

MCP Tools

外部工具链

Self-Heal & Retry

崩溃自恢复

Security Gates

危险操作拦截

Job Scheduler

后台调度

可以外购 · BUY

底盘 + 手脚

Kiro CLI · AgentCore · Claude Code · Codex · 开源框架

只能自建 · BUILD

大脑 · 知识层

买不到、抄不走 —— 只能一次次用出来

所以只做一件事

建 Knowledge Layer

主题 4 · STAY SAFE

Full-stack Engineer · EE / OE

怎么守住 AI Coding 的质量,跟上 AI Coding 的速度

AIDLC · Agent 进入每一环 Execution —— 人的 Judgement 反而越来越重要

Engineer 不会被取代 —— 工程能力 · 系统化思考 · judgement 只会更稀缺。但依然有优化空间



这就是我们上生产的真实链路 —— 11 个 stage 一个都砍不掉。我们要做的:每一环尽可能自动化、让 AI 执行,把 人的判断力留给真正重要的裁决。

SO THE REAL QUESTION Human in the Agent Loop or Agent in the Human Loop?

AIDLC 让交付更快——可质量与安全的 P0 指标却在破防

速度兑现了,但底线守不住,速度就毫无意义。

瓶颈没有消失,它转移了 BOTTLENECK SHIFT Agent 解决中间 coding(~30% 链路),瓶颈移向 两头 —— EE/OE 恰住在这里

Shift Left ←

定义 + 协调

需求 · Spec · 跨角色对齐 —— 新瓶颈 · EE 战场

✓ Solved by Agent

Coding

曾经的瓶颈,如今 绝大多数改动由 coding-agent 产出。

~30% 链路

Shift Right →

质量 + 安全

验证 · 回归 · Security —— 新瓶颈 · OE 战场

EE/OE P0 指标趋势 Productivity(ND)与质量(Ticket)一路走高,但安全 High findings 近期数倍骤增 —— 破防



Amazon 靠 EE/OE 来确保"造得对"和"运营好"

EE 管"造得对"——写代码前和写代码时的工程标准;OE 管"运营好"——上线之后的运营承诺。

OE Operational Excellence · 运营好

"You build it, you run it" · 上线之后

EE Engineering Excellence · 造对

Learn and Be Curious + Insist on the Highest Standards · 写代码前 + 写代码时

把"造对"标准化、可培训、可度量的能力体系

"Everything fails, all the time." —— OE 不追求不出事,而是**为失败而设计**,把恢复力、可观测性、自动化当一等公民。
— Werner Vogels

OE 权威定义

"Operational Excellence is a commitment to **build software correctly** while consistently delivering a **great customer experience**."

- 1 Design & build**
架构即可靠
- 2 Operate**
oncall + 变更管理
- 3 Test & verify**
持续验证
- 4 Improve w/ metrics**
数据驱动

	EE — 造对	OE — 运营好
阶段	写代码前 + 写代码时	上线之后
关键动作	design review · 测试 · CI/CD · 自动代码审查	oncall · 监控告警 · 事故复盘 · 持续改进
载体	技能培养 · 知识库 · AI Fluency	年度规划 · 上线评审 · 事故复盘 · 架构最佳实践
心法	Insist on the Highest Standards	You build it, you run it

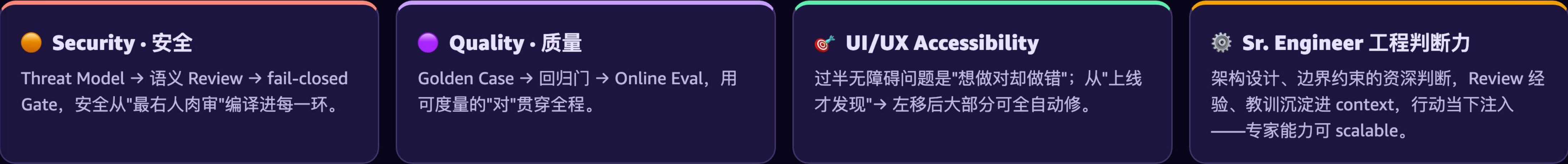
2026
新前沿

- **EE** 造对被 AI 重写 — 人写代码 → 人监督 agent 交付 · AIDLC · AI Fluency
- **OE** 运营被 AI 重写 — 人 oncall → Security / DevOps Agent 值守 · SecDLC · Findings → Triage → Fixing · 人监督

EE/OE — 编译进 AI-DLC 的每一环

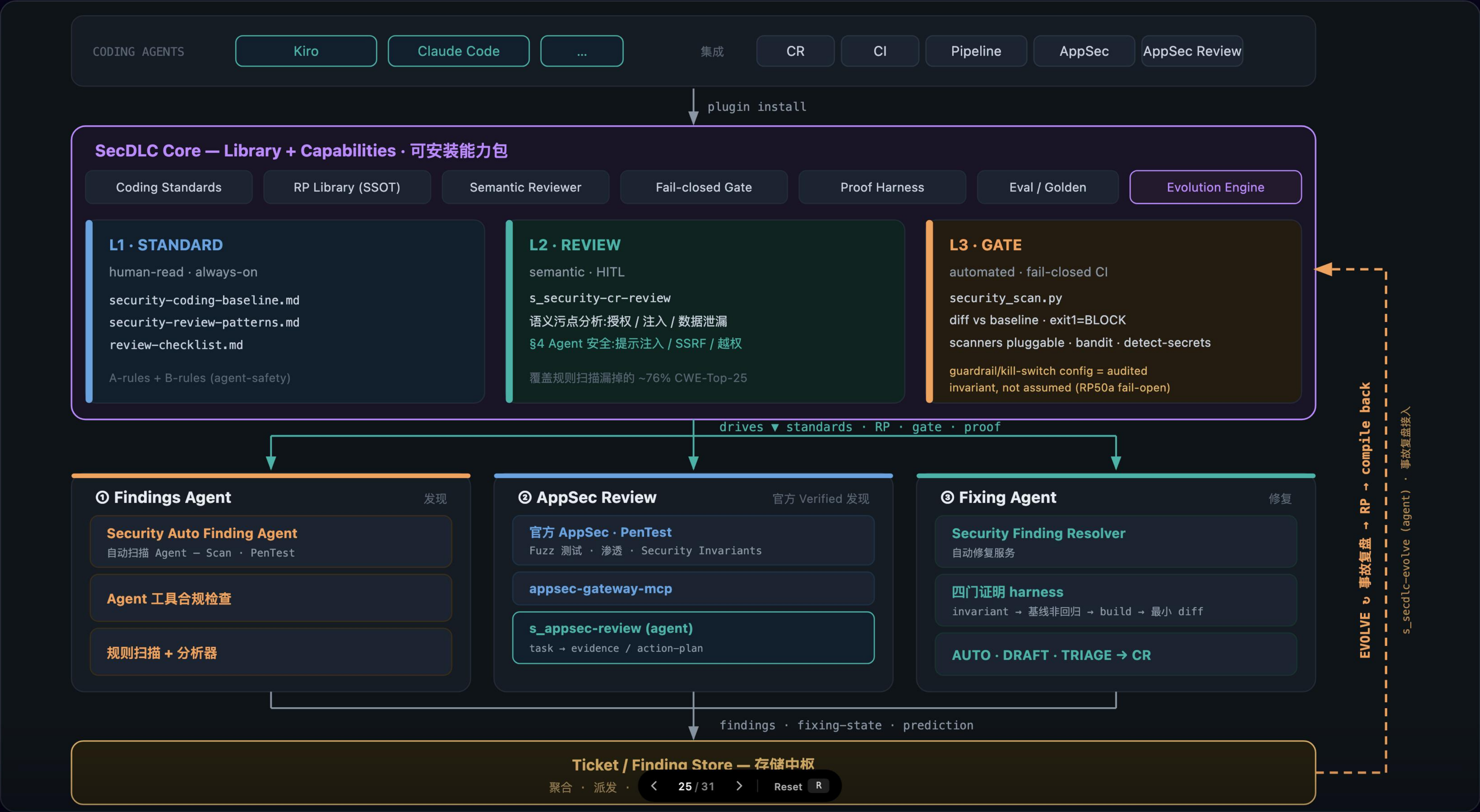
从需求到运营,每个 stage 都有对应的 EE/OE 动作、产物与 Gate。

← Shift-Left · EE 造对 ————— OE 运营 · Shift-Right →



Sample-1:把安全编译进开发生命周期 - SecDLC

用 AI-Native 方式守住 AI 时代的安全底线。



Sample-2:怎么做好 Agent 产品的测试 - Eval-First

用 Golden Case 贯穿全生命周期持续自证。



<

26 / 31

>

Reset

R

7×24 AIxEngineers - 把部分 Operations 交给 AI 工程师

AutoOps / AutoSecurity 是「start autonomous」的首选切入点 —— 输入边界清晰、有安全网,风险可控、ROI 清晰;开放式的人指挥 Coding 通道反而最后才收敛全自动。

 **Human Take Over** — 任何环节,人都可随时接管

人指挥
judge & 精力



 **+Coding Agent** 人指挥
Steering Agent 干活 —— Produce Coding

PRs

Tickets
bugs / issues



 **AutoOps Agent** 7×24 全自动
定位、修复 bug/issue,协助 on-call triage

PRs

Fixing Instr.

Security
Findings



 **AutoSecurity Agent** 7×24 全自动
处理 security findings,能确诊的修、存疑的标

PRs

Fixing Instr.

 **统一审查**

AutoPRReviewer

所有 PR 逐条审 · 参考工程标准

AutoSecReviewer

PR 的安全审查 ·
SecDLC 语义 review

不管来源,先过审再到人



 **人**

- Review
- Comments
- Approve
- Judge
- Triage
- Take Over

收到:已审 PRs +
Fixing Instr.
需介入 → 回 
+Coding Agent

 **Comments → Revise / Address Comments** —— AutoReviewer 或人的 comments 回流到对应 Agent,revise 后重新提交、再审

◀ 回左侧 **Agent**

 **Evolution**

 **Project / Product Context**

DDD · System Specs · Code Graph · Source Repo / Packages



 **Domain Expert Knowledge**

Security · Bug Fixing · 事故复盘 · Lessons & Learns

**复利 →
驱动下一次 fixing**

主题 5 · THEN ...

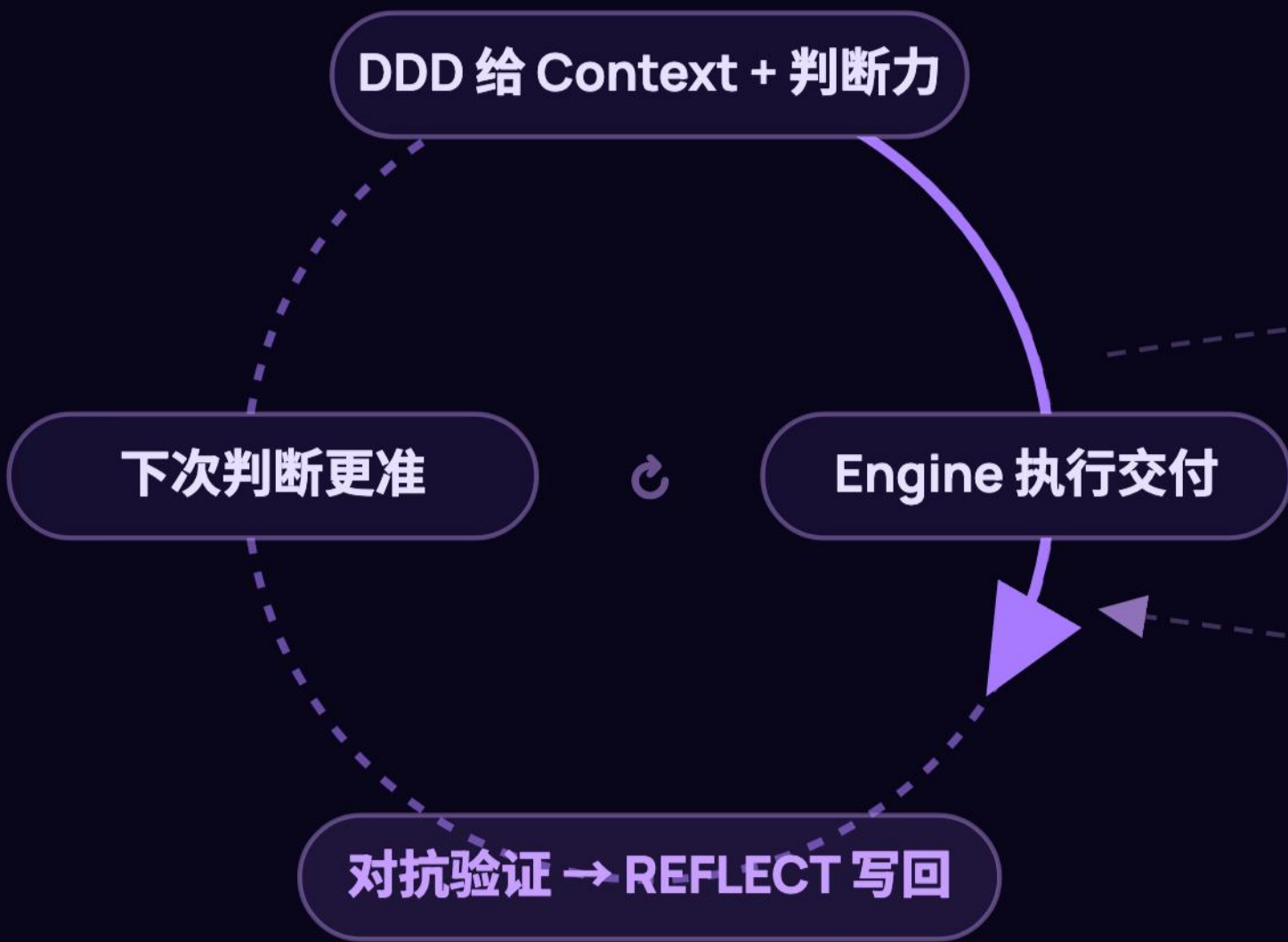
Then ...

守住了底线 —— 技术复利 × 组织复利, 飞轮转起来

双飞轮——技术复利 × 组织复利

飞轮 A · 系统越用越聪明

技术复利 —— 犯过的错，不再犯第二次



飞轮 B · 团队越用越强

组织复利 —— 一人的发现，惠及所有人的 Agent



系统更聪明 → 团队用得更顺



互相加速

更多执行 → 更多 REFLECT 写回

你现在用的 AI，犯过的错会不会再犯第二次？ 会 → 它是工具；不会 → 它在 **compound**。

没飞轮：第 1 天 2x，第 300 天还是 2x（租工具） | 有飞轮：第 1 天 2x，第 300 天 **10x**（建资产）

复利循环本身，就是产品。

一个 Builder，服务两条线



从今开始——Build 你的复利循环，让飞轮转起来

1 先诊断,建地基

用 S×T 矩阵定位自己在哪一格 · 定义
PO Metrics (Output / Outcome,不量
input)

支柱 1-3:文化 · 度量 · 就绪度

→ 2 SDD → Autonomous

Spec 即合约,全链条上线 · 判断力沉进
DDD,Agent 才敢自主

AI-Ready Repo · 9 Stages · 3 Gates

→ 3 守住底线

EE / OE 编译进每一环 · SecDLC +
Eval-First 与速度同步

守不住底线,速度毫无意义

→ 4 让飞轮转

每次 REFLECT 写回 DDD —— 技术复
利 × 组织复利

第 300 天不是 2x,是 10x

Human Directs, AI Delivers.